SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

NAME: M Manoj Reddy **REG NO:** 192372282

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: 2 **Duration : 45 Minutes**

Total Marks: 20

Annexure A

Descriptive Written Test

Code of Conduct:

*I, **M Manoj Reddy (192372282)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Manoj Reddy

Question 1: Intelligent Agent Types in a Smart Home Automation System

Objective

To explain the four classical types of intelligent agents — simple reflex, model-based, goal-based, and utility-based — and to analyse how each would function within a smart home automation system that manages lighting, temperature, and security, and to justify which type or hybrid combination is most effective for comfort, energy efficiency, and security.

Problem Statement

A smart home automation system must manage lighting, temperature, and security based on user behaviour and environmental conditions. It must respond to real-time inputs — motion detection, temperature changes, and time of day — while also learning user preferences over time. The task is to explain how each agent type would function in this system and justify the most effective design.

Solution — Agent Types Explained

1. Simple Reflex Agent

Acts purely on the current percept using fixed condition-action rules, with no memory of the past and no model of the world. In the smart home: "IF motion detected THEN turn lights ON", "IF temperature > 26°C THEN turn AC ON". It is fast and simple but reacts blindly — for example, it turns lights off the instant motion briefly stops, even if the room is still occupied, and it cannot anticipate patterns like a user's regular evening routine.

2. Model-Based Reflex Agent

Maintains an internal state/model of the world that is updated as new percepts arrive, so it can handle partial observability. In the smart home, it might remember recent occupancy over the last few minutes (so lights don't flicker off the instant someone sits still) or track a running average room temperature to smooth out noisy sensor spikes. This produces steadier, more realistic behaviour than the purely reactive simple reflex agent.

3. Goal-Based Agent

Chooses actions by reasoning about which action sequence achieves an explicit goal, rather than just reacting to conditions. In the smart home, goals could be "keep the room temperature within 21–24°C" or "ensure the home is secured whenever no one is present". This lets the agent plan proactively — e.g., pre-cooling a room before a user is expected home — rather than only reacting after a threshold is crossed.

4. Utility-Based Agent

Extends the goal-based agent by assigning a utility (desirability) score to different ways of achieving a goal, allowing it to choose the best option when multiple goals compete — e.g., balancing comfort against energy cost. In the smart home, a utility-based agent could weigh comfort, energy savings, and security together and pick the overall best action (e.g., accept slightly less comfort for meaningfully lower energy use when the home is unoccupied).

Justification: A Hybrid Utility-Based / Learning Agent is Most Effective

No single classical type alone is sufficient. Simple reflex behaviour is needed for instant safety responses (e.g., smoke detected → alarm immediately), but relying only on reflex rules causes erratic comfort behaviour. A model-based layer is essential simply to handle sensor noise and short gaps in occupancy sensing. A goal-based layer is needed to pursue explicit objectives such as temperature targets and security arming/disarming. Finally, because comfort, energy efficiency, and security often conflict (e.g., maximum comfort usually costs more energy), a utility-based layer is required to make the actual trade-off decision. Adding a learning component on top (adjusting rules, models, and utility weights from observed user behaviour) allows the system to personalise itself over time. The most effective design is therefore a **hybrid, learning, utility-based agent** that uses simple reflex rules only for safety-critical instant responses, a model-based internal state for smoothing sensor data, explicit goals for comfort/security targets, and a utility function to resolve trade-offs between them.

Code (Simulating All Four Agent Types on the Same Sensor Readings)

```
readings = [
    {"time": "07:00", "motion": True,  "temp": 19, "user_home": True},
    {"time": "09:30", "motion": False, "temp": 22, "user_home": False},
    {"time": "13:00", "motion": False, "temp": 29, "user_home": False},
    {"time": "18:45", "motion": True,  "temp": 24, "user_home": True},
    {"time": "23:30", "motion": False, "temp": 21, "user_home": True},
]

# 1. Simple Reflex Agent -- fixed condition-action rules only
for r in readings:
    actions = ["Lights ON" if r["motion"] else "Lights OFF"]
    if r["temp"] > 26: actions.append("AC ON")
    elif r["temp"] < 20: actions.append("Heater ON")
    print(r["time"], "->", actions)

# 2. Model-Based Reflex Agent -- keeps recent-occupancy + running-avg temp state
occupancy_history, temp_history = [], []
for r in readings:
    occupancy_history.append(r["motion"]); temp_history.append(r["temp"])
    recently_occupied = any(occupancy_history[-3:])
    avg_temp = sum(temp_history) / len(temp_history)
    ...  # decide lights/HVAC using internal state, not just current percept

# 3. Goal-Based Agent -- pursues explicit goals
for r in readings:
    if r["temp"] < 21: action = "Heater ON (goal: 21-24C)"
    elif r["temp"] > 24: action = "AC ON (goal: 21-24C)"
    else: action = "Temperature OK"
    security = "ARMED" if not r["user_home"] else "DISARMED"

# 4. Utility-Based Agent -- scores competing modes, picks the best
def utility(comfort, energy_saving, security):
    return 0.4*comfort + 0.3*energy_saving + 0.3*security
# compute utility for Comfort-priority / Energy-saving / Security-priority modes
# and select the option with the highest score for each reading
```

Output (Illustrative Simulation Results)

```
SIMPLE REFLEX AGENT (reacts only to current percept):
07:00: motion=True,  temp=19C -> Lights ON, Heater ON
09:30: motion=False, temp=22C -> Lights OFF
13:00: motion=False, temp=29C -> Lights OFF, AC ON
18:45: motion=True,  temp=24C -> Lights ON
23:30: motion=False, temp=21C -> Lights OFF

MODEL-BASED REFLEX AGENT (remembers last 3 readings + running avg temp):
07:00: -> Lights ON (recent activity), Heater ON (avg 19.0C)
09:30: -> Lights ON (recent activity -- smoother than reflex agent)
13:00: -> Lights ON (recent activity)
18:45: -> Lights ON (recent activity)
```

```
23:30: -> Lights ON (recent activity)

GOAL-BASED AGENT (goal: temp in 21-24C, security armed when empty):
07:00: -> Heater ON -> target 21-24C, Security DISARMED (user present)
09:30: -> Temperature OK, Security ARMED (protect empty home)
13:00: -> AC ON -> target 21-24C, Security ARMED (protect empty home)
18:45: -> Temperature OK, Security DISARMED (user present)
23:30: -> Temperature OK, Security DISARMED (user present)

UTILITY-BASED AGENT (scores Comfort / Energy-saving / Security modes):
07:00: BEST = Security-priority mode   (utility 6.10)
09:30: BEST = Energy-saving mode       (utility 7.70)
13:00: BEST = Energy-saving mode       (utility 7.70)
18:45: BEST = Comfort-priority mode    (utility 6.40)
23:30: BEST = Comfort-priority mode    (utility 6.40)
```

Discussion

The simulation highlights the practical differences clearly. The simple reflex agent switches lights off the instant motion pauses (09:30, 13:00, 23:30), which would feel jarring to a real user. The model-based agent instead keeps lights on based on recent activity, producing smoother, less flickery behaviour. The goal-based agent's decisions are easy to justify in plain language ("heater on because temperature is below the 21–24°C goal"), which is valuable for explainability. The utility-based agent shows the clearest adaptive intelligence: at 09:30 and 13:00, when the home is empty, it automatically favours the energy-saving mode over comfort, while at 18:45 and 23:30, when the user is present, it switches its preference to comfort — exactly the kind of context-aware trade-off a purely rule-based agent cannot achieve.

Conclusion

Each classical agent type contributes something the others lack: simple reflex for instant safety-critical reactions, model-based reasoning for stability against noisy or intermittent sensing, goal-based reasoning for pursuing explicit comfort/security objectives, and utility-based reasoning for resolving conflicts between comfort, energy efficiency, and security. A hybrid, learning, utility-based architecture that combines all four layers is therefore the most effective design for a real smart home automation system.

Question 2: Robot in an Unknown Maze — UCS and IDS

Objective

To explain how the uninformed search strategies Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can guide a robot to find an exit path in an unknown maze, compare their time and space complexity, relate the problem to intelligent agent types, and recommend the most effective combination of agent design and search strategy.

Problem Statement

A robot is placed in an unknown maze and must find a path to the exit without any prior knowledge of the maze's layout. It must decide, step by step, which uninformed search strategy to use, and how its own agent architecture (e.g., simple reflex vs. model-based) affects its ability to explore and navigate effectively.

Solution — How UCS and IDS Apply

Uniform Cost Search (UCS) expands the frontier node with the lowest cumulative path cost g(n) first, using a priority queue. In the maze, if every move costs 1, UCS behaves like Breadth-First Search and finds the shortest path to the exit; if some cells (e.g., rough terrain) cost more to traverse, UCS still finds the genuinely cheapest route, not just the fewest-steps route. **Iterative Deepening Search (IDS)** repeatedly performs a depth-limited Depth-First Search with an increasing depth limit (0, 1, 2, ...) until the exit is found. Each iteration re-explores from scratch, but because it goes only one level deeper each time, it still finds the shallowest (shortest-step) exit path, while using far less memory than a search that must store the entire frontier.

Complexity Comparison

Aspect	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Time complexity	O(b^(1+floor(C*/e))) -- depends on the cost of the optimal path C* and smallest edge cost e	O(b^d) -- d = depth of the shallowest goal; some overhead from repeated shall
Space complexity	O(b^(1+floor(C*/e))) -- must store the whole frontier (by priority queue)	O(bd) -- only stores the current path (depth-first), far mo
Completeness	Complete (finds a solution if one exists, assuming positive edge costs)	Complete for finite mazes
Optimality	Optimal -- always finds the least-cost path	Optimal for unit-cost mazes (finds shortest-step path)
Best suited when...	Move costs vary (e.g., rough terrain, energy costs differ by cell)	Memory is tightly constrained (e.g., a small embedded robot cont

Advantages and Disadvantages

- **UCS advantages:** guarantees the true minimum-cost path even with variable edge costs; systematic and complete.
- **UCS disadvantages:** can consume large amounts of memory, since it must keep the entire frontier in a priority queue -- potentially problematic for a large maze on a resource-constrained robot.
- **IDS advantages:** memory usage is linear in path depth, making it practical for large or unknown-size mazes on limited-memory hardware; still guaranteed to find the shortest path in a unit-cost maze.
- **IDS disadvantages:** repeats work at every depth iteration, so its total node expansions are typically higher than UCS/BFS for the same problem, trading time for memory savings.

Relating the Problem to Intelligent Agent Types

A **simple reflex agent** cannot solve this maze at all beyond trivial cases -- with no memory of where it has already been, it can easily loop forever bouncing between the same cells. A **model-based reflex agent** that maintains a visited-cells map is the minimum required just to guarantee termination and avoid revisiting dead ends. Because the robot needs to actively construct and follow a path to a specific goal (the exit), it must ultimately be a **goal-based agent**, using its internal maze model together with a search algorithm (UCS or IDS) to plan the path. If different maze cells carry different traversal costs (e.g., rubble, slopes, or energy use), a **utility-based agent** becomes relevant too, since it can weigh path cost against exploration risk or time pressure. In short, the search strategy is the planning mechanism the goal-based agent uses internally, and the surrounding agent architecture determines whether that plan can even be constructed and executed reliably.

Recommended Combination and Justification

The most effective design is a **model-based, goal-based agent using IDS as its default search strategy, with a fallback to UCS when move costs are non-uniform**. IDS is the right default for an unknown maze because the robot has no prior size or memory budget guarantee -- IDS explores with only O(b·d) memory while still guaranteeing the shortest path in the common case of unit-cost moves. If the maze is known to have varying terrain costs, the agent should switch to UCS, since only UCS guarantees true cost-optimality when costs are unequal. In both cases, the goal-based agent's internal (model-based) map of visited cells and known walls is essential; without it, neither search algorithm could be executed as the robot physically moves and senses one step at a time, since the robot can only discover the maze structure incrementally rather than seeing the entire layout upfront.

Code (Python implementation of UCS and IDS on a sample maze)

```
import heapq

maze = [   # 0 = free cell, 1 = wall
    [0,0,0,1,0,0,0],
    [1,1,0,1,0,1,0],
    [0,0,0,1,0,1,0],
    [0,1,1,1,0,1,0],
    [0,0,0,0,0,1,0],
    [0,1,1,1,1,1,0],
    [0,0,0,0,0,0,0],
]
START, GOAL = (0,0), (6,6)

def neighbors(pos):
    r,c = pos
    for nr,nc in [(r-1,c),(r+1,c),(r,c-1),(r,c+1)]:
        if 0<=nr<len(maze) and 0<=nc<len(maze[0]) and maze[nr][nc]==0:
            yield (nr,nc)

# ---- Uniform Cost Search ----
def ucs(start, goal):
    frontier = [(0, start, [start])]
    visited, nodes_expanded = set(), 0
    while frontier:
        cost, node, path = heapq.heappop(frontier)
        if node in visited: continue
        visited.add(node); nodes_expanded += 1
        if node == goal:
            return path, cost, nodes_expanded
        for n in neighbors(node):
            if n not in visited:
                heapq.heappush(frontier, (cost+1, n, path+[n]))
    return None, None, nodes_expanded
```

```
# ---- Iterative Deepening Search ----
def dls(node, goal, limit, path, seen, stats):
    stats['nodes'] += 1
    if node == goal: return path
    if limit <= 0: return None
    for n in neighbors(node):
        if n not in seen:
            result = dls(n, goal, limit-1, path+[n], seen | {n}, stats)
            if result: return result
    return None

def ids(start, goal, max_depth=40):
    total = 0
    for depth in range(max_depth+1):
        stats = {'nodes': 0}
        result = dls(start, goal, depth, [start], {start}, stats)
        total += stats['nodes']
        if result:
            return result, len(result)-1, total, depth
    return None, None, total, max_depth

path_ucs, cost_ucs, exp_ucs = ucs(START, GOAL)
path_ids, cost_ids, exp_ids, depth = ids(START, GOAL)
print("UCS  -> cost:", cost_ucs, "nodes expanded:", exp_ucs)
print("IDS  -> cost:", cost_ids, "nodes expanded:", exp_ids)
```

Output — UCS vs. IDS on the Sample Maze

```
UNIFORM COST SEARCH (UCS):
Path: (0,0)->(0,1)->(0,2)->(1,2)->(2,2)->(2,1)->(2,0)->(3,0)->(4,0)
      ->(5,0)->(6,0)->(6,1)->(6,2)->(6,3)->(6,4)->(6,5)->(6,6)
Path cost: 16
Nodes expanded: 25

ITERATIVE DEEPENING SEARCH (IDS):
Path: (0,0)->(0,1)->(0,2)->(1,2)->(2,2)->(2,1)->(2,0)->(3,0)->(4,0)
      ->(5,0)->(6,0)->(6,1)->(6,2)->(6,3)->(6,4)->(6,5)->(6,6)
Path cost (depth): 16
Total nodes expanded (summed across all depth iterations 0..16): 181
Depth limit at which goal was found: 16

COMPARISON:
Metric                          UCS        IDS
Path cost                        16         16
Nodes expanded                   25        181
```

Discussion

Both algorithms find the same optimal 16-step path, confirming that for this unit-cost maze IDS matches UCS's optimality guarantee. However, the node-expansion counts make the time/space trade-off concrete: UCS expanded only 25 nodes in total (since it never revisits a node once settled), whereas IDS's cumulative node count across all 17 depth iterations (0 through 16) reached 181, because each new depth limit re-explores much of the same shallow territory again. In exchange, IDS never needed to hold more than a single root-to-current-node path in memory at any point, while UCS's priority queue must retain every unexpanded frontier node simultaneously. This experiment mirrors the general textbook result: IDS trades additional computation time for substantially lower memory use compared to UCS/BFS-style search.

Conclusion

Both UCS and IDS successfully guide the maze-solving robot to the exit via the optimal 16-step path. UCS is more time-efficient when memory is not a constraint or when move costs vary, while IDS is far more memory-efficient at the cost of repeated node expansions. Given that a real maze-exploring robot typically has to combine limited onboard memory with a genuinely unknown environment, the most effective overall design is a model-based, goal-based agent that defaults to IDS for its favourable memory profile, switching to UCS only when the maze is known to have non-uniform traversal costs.